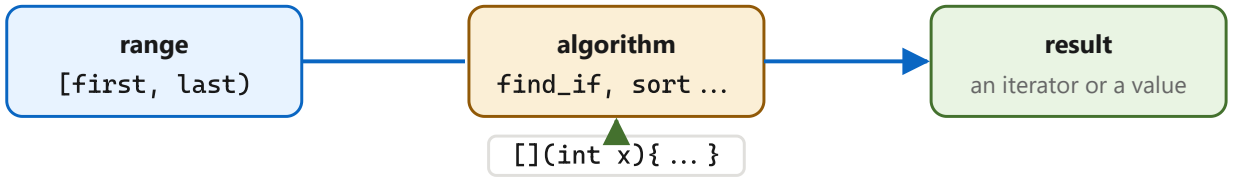
Algorithms and Ranges

Prefer named algorithms to hand-written loops. sort, find and transform with lambdas, the erase-remove idiom, binary search, C++20 ranges and views, and the random library. Assumes Iterators.

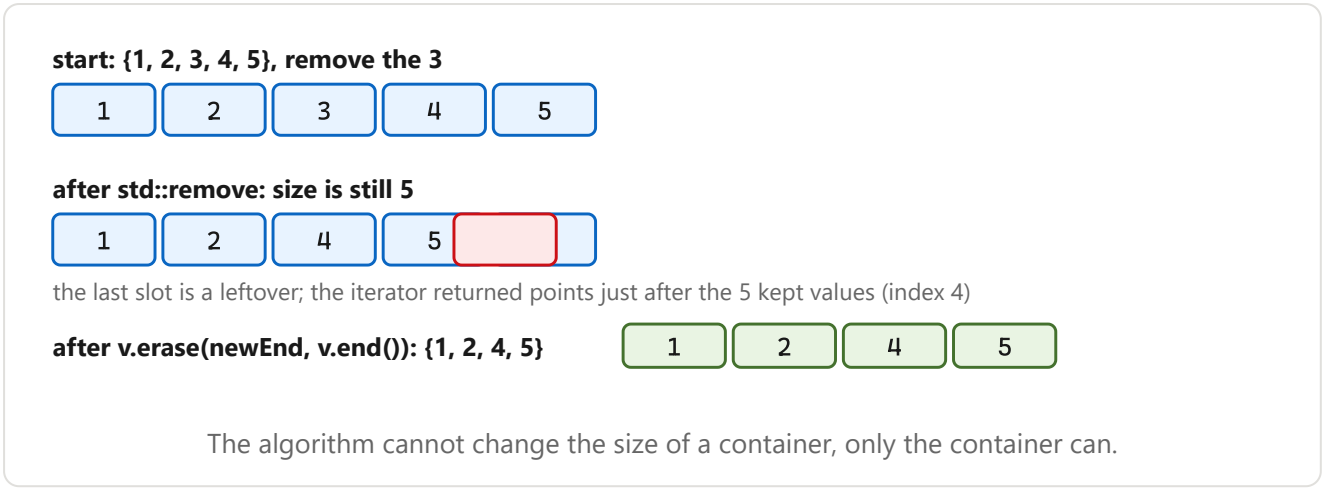
Algorithms

Over a hundred generic functions in `<algorithm>` / `<numeric>`. Prefer them to loops: named, tested, often faster. Most take a lambda.

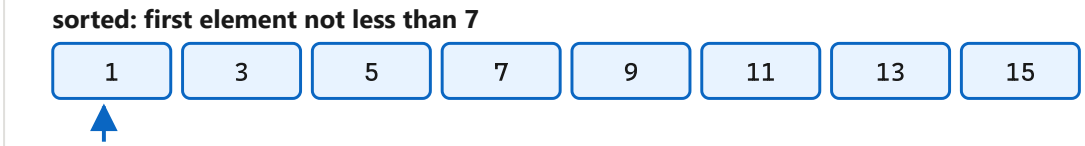


The lambda decides what counts as a match, an order or a transformation.

```
// custom order
std::sort(v.begin(), v.end(), [](const P& a, const P& b) { return a.age <
b.age; });
auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 6; });
int n  = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
bool any = std::any_of(v.begin(), v.end(), [](int x) { return x < 0; });
int sum  = std::accumulate(v.begin(), v.end(), 0);          // <numeric>
std::transform(v.begin(), v.end(), v.begin(), [](int x) { return x * x; });
// binary search: range MUST be sorted
auto pos = std::lower_bound(sorted.begin(), sorted.end(), 5);
```



The algorithm cannot change the size of a container, only the container can.



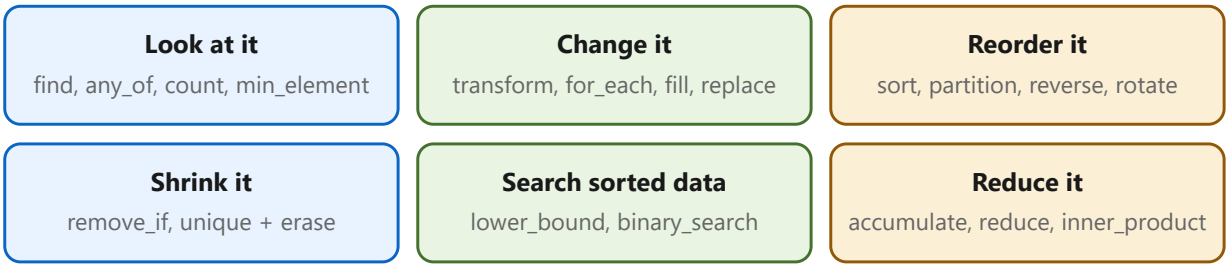
1. look at the middle; 2. keep one half; 3. repeat: log2(n) steps, but only if the range is sorted

A binary search on unsorted data returns nonsense.

STRICT WEAK ORDERING Use `<`, never `≤`: a comparator true for equal elements can crash `sort` or corrupt a `set`. `sort` is not stable; use `stable_sort`.

Which algorithm do I need?

There are over a hundred algorithms, but they fall into a few families. Ask what you want from the range, then pick the family.



Name the question first; the algorithm usually has the obvious name.

YOU WANT TO...	USE	NOTES
know if any, all or none match	`any_of`, `all_of`, `none_of`	stop at the first answer
find the first match	`find`, `find_if`	returns `end()` when nothing matches
apply a function to every element	`for_each`, `transform`	`transform` writes results to an output range
put matching elements first	`partition`, `stable_partition`	returns the split point
get the k largest in order	`partial_sort`, `nth_element`	cheaper than a full sort
remove duplicates	`sort`, then `unique` and `erase`	`unique` only removes **neighbouring** duplicates
sum or fold	`accumulate`	the start value decides the result type: write `0.0` for doubles

```
std::vector<int> v{5, 2, 9, 2, 7, 4}, sq;
// sq: 25 4 81 4 49 16
std::transform(v.begin(), v.end(), std::back_inserter(sq), [](int x) { return
x * x; });

auto mid = std::partition(v.begin(), v.end(), [](int x) { return x % 2 == 0;
});
// the evens come first; mid - v.begin() is 3 here (compiled)

std::sort(v.begin(), v.end());
// {2, 4, 5, 7, 9}
v.erase(std::unique(v.begin(), v.end()), v.end());
// 5.4 (compiled)
double avg = std::accumulate(v.begin(), v.end(), 0.0) / v.size();
```

Ranges and views (C++20)

```cpp
#include <ranges>

// pass the container, not begin() and end()
std::ranges::sort(v);
// descending (compiled: first element 6)
std::ranges::sort(v, std::greater<>{});

auto evenSquares = v | std::views::filter([](int x) { return x % 2 == 0; })
                     // a LAZY view: nothing is computed yet
                     | std::views::transform([](int x) { return x * x; });
// computed on demand, no temporary vector (compiled: sum 56)
for (int x : evenSquares) use(x);
```

- A **range** is anything with `begin()` and `end()`. The `std::ranges::` algorithms take the whole range and support **projections**: `std::ranges::sort(people, {}, &Person::age)`.
- A **view** is a lightweight, non-owning, lazy window (`filter`, `transform`, `take`, `drop`, `reverse`, `iota`). Pipe them with `|`.
- A view does not own its data: if the container dies or reallocates, the view dangles, like `string_view`.
- Ranges make pipelines readable; you do not need to know every view.

Common algorithm pitfalls

PITFALL	WHY	FIX
`remove` without `erase`	algorithms cannot change the size of a container	erase-remove, or `std::erase_if` (C++20)
writing into an empty output with `begin()`	the algorithm assigns into slots that do not exist	`std::back_inserter(out)`
`binary_search` or `lower_bound` on unsorted data	the halving logic assumes order	sort first, with the same comparison
`unique` on unsorted data	it removes only neighbouring duplicates	`sort` first
comparator using `≤`	not a strict weak ordering: undefined behavior	use `<`
`accumulate( … , 0)` over doubles	the int start value truncates every partial sum	start with `0.0`
a lambda that captures a dangling reference	the algorithm runs while the lambda is alive, but a stored one may outlive its data	capture by value when the lambda is stored
`std::find` on a `map`	it walks every pair in O(n)	use the member `m.find(key)`

Random numbers: `<random>`

```cpp
// an ENGINE: produces raw random bits (here seeded for repeatability)
std::mt19937 gen(12345);
// a DISTRIBUTION: shapes the bits into the numbers you want
std::uniform_int_distribution<int> dice(1, 6);
int roll = dice(gen);

// a non-deterministic seed source (hardware or OS)
std::random_device rd;
std::mt19937 gen2(rd());
std::normal_distribution<double> noise(0.0, 1.0);
```

- **Engine plus distribution**: never use `rand() % n` (poor quality, biased, global state). Same seed gives the **same sequence** (compiled: true), which is what you want in tests and simulations.
- Compiled: 6000 rolls of the dice gave every face between 850 and 1150 times; 10,000 normal samples had a mean near 0.
- Keep one engine per thread; engines are not thread-safe.

Notes: Module 3.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Algorithms	C++ Primer	10.1 to 10.2 Overview and a first look	405 to 413
	Programming: Principles and Practice	21.1 to 21.2, 21.5 Standard algorithms, function objects, sorting and searching	617 to 626 and 633 to 635
	C++ Move Semantics	Chapter 14 Moving algorithms and iterators	219 to 228
Random numbers	C++ Primer	17.4 Random numbers	774 to 780